

Exercícios de Fluxo de Controle

Grafo de Fluxo de Controle (CFG) e Complexidade Ciclomática

Resolução dos exercícios propostos, com a construção dos grafos de fluxo de controle, cálculo da complexidade ciclomática pelas duas fórmulas apresentadas e definição de uma base de caminhos independentes para cada método.

Exercício 1 — Classificação de pedido

Código analisado

```
public String classificarPedido(
    double valor,
    boolean clienteVip,
    boolean pagamentoAprovado) {

    double desconto = 0;

    if (valor >= 500) {
        desconto = 10;
    }

    if (clienteVip) {
        desconto += 5;
    }

    if (!pagamentoAprovado) {
        return "PAGAMENTO RECUSADO";
    }

    double valorFinal = valor - (valor * desconto / 100);
    return "PEDIDO APROVADO: " + valorFinal;
}
```

1) Blocos básicos

O método foi dividido em 7 blocos de instrução (mais um nó final de encerramento, usado só pra juntar os dois pontos de saída do método):

B1 — desconto = 0; e teste valor >= 500

B2 — desconto = 10; (ramo verdadeiro do 1º if)

B3 — teste clienteVip (ponto de junção depois do 1º if)

B4 — desconto += 5; (ramo verdadeiro do 2º if)

B5 — teste !pagamentoAprovado (junção depois do 2º if)

B6 — return "PAGAMENTO RECUSADO"; (ramo verdadeiro do 3º if)

B7 — valorFinal = valor - (valor * desconto / 100); e return "PEDIDO APROVADO...";

FIM — nó de saída único do método (une B6 e B7)

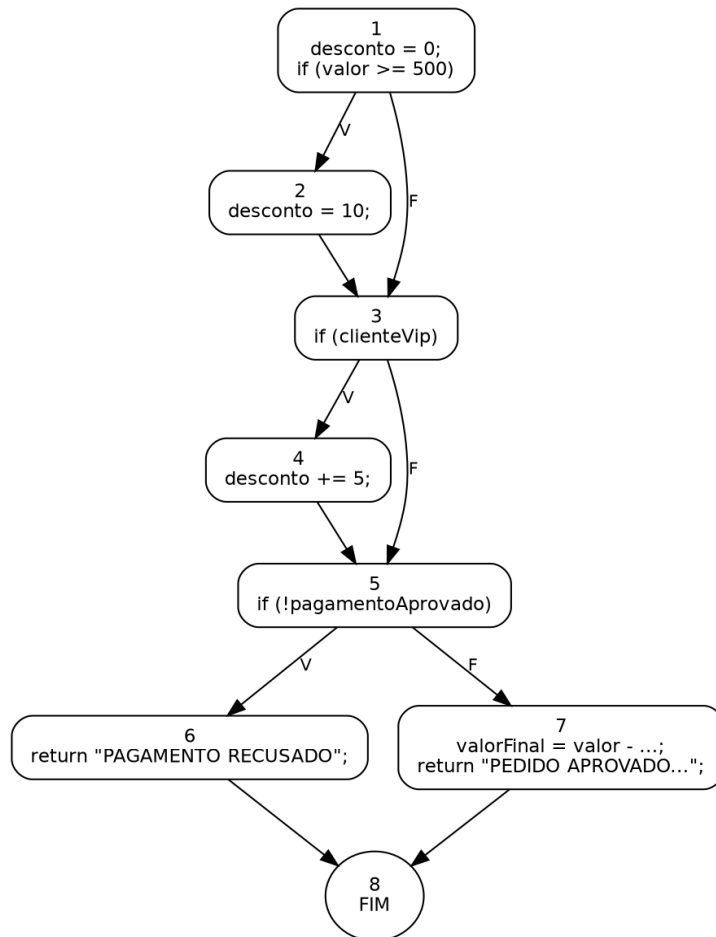
2) Decisões identificadas

São três decisões independentes, uma depois da outra (nenhuma está aninhada dentro da outra):

- D1: **valor >= 500**
- D2: **clienteVip**
- D3: **!pagamentoAprovado**

3) e 4) Grafo de Fluxo de Controle

O return dentro do terceiro if aparece no grafo como uma aresta que sai direto do bloco 6 para o nó final, sem passar pelo bloco 7 — ou seja, quando o pagamento não foi aprovado, o cálculo de valorFinal simplesmente não é alcançável.



Grafo de fluxo de controle do método `classificarPedido` (V = verdadeiro, F = falso).

5) e 6) Contagem de nós, arestas e complexidade ciclomática

N (número de nós) = 8

E (número de arestas) = 10

$$V(G) = E - N + 2 = 10 - 8 + 2 = 4$$

7) Conferência pela segunda fórmula

$$V(G) = \text{número de decisões} + 1 = 3 + 1 = 4$$

Os dois cálculos batem, então o grafo está consistente.

8), 9) e 10) Base de caminhos independentes

Como $V(G) = 4$, a base precisa ter 4 caminhos, e cada um deles tem que trazer pelo menos uma aresta nova que ainda não apareceu nos caminhos anteriores. Segue a base escolhida:

- **P1** (nós 1-2-3-4-5-6-FIM) — entrada: valor = 600, clienteVip = true, pagamentoAprovado = false. Resultado esperado: "PAGAMENTO RECUSADO"
- **P2** (nós 1-3-4-5-6-FIM) — entrada: valor = 100, clienteVip = true, pagamentoAprovado = false. Resultado esperado: "PAGAMENTO RECUSADO"
- **P3** (nós 1-3-5-6-FIM) — entrada: valor = 100, clienteVip = false, pagamentoAprovado = false. Resultado esperado: "PAGAMENTO RECUSADO"
- **P4** (nós 1-3-5-7-FIM) — entrada: valor = 100, clienteVip = false, pagamentoAprovado = true. Resultado esperado: "PEDIDO APROVADO: 100.0"

Conferindo: P1 usa as arestas 1→2 e 2→3 que nenhum outro caminho usa; P2 usa a aresta 1→3 pela primeira vez; P3 usa a aresta 3→5 pela primeira vez; e P4 usa a aresta 5→7 (a única que leva ao

cálculo de valorFinal). Juntando os quatro caminhos, as 10 arestas do grafo acabam todas cobertas.

Questões para discussão

Quantas combinações entre as três condições são possíveis? Como são três condições booleanas independentes, dá $2^3 = 8$ combinações possíveis de entrada.

O número de combinações é igual à complexidade ciclomática? Não. A complexidade ciclomática (4) conta uma base de caminhos linearmente independentes no grafo, não o total de execuções possíveis. Repare que, no grafo, existem 8 rotas distintas entre o nó 1 e o nó FIM (uma para cada combinação das três condições), mas várias delas reaproveitam o mesmo conjunto de arestas já contado em outra rota — por isso a base mínima fica em 4, mesmo havendo 8 combinações de entrada possíveis.

Como o return dentro da terceira condição altera o grafo? Ele cria uma saída antecipada: o bloco 6 vai direto para o nó de encerramento, sem passar pelo bloco 7. Se não existisse esse return, o grafo seria uma sequência só, sem esse desvio de saída no meio do fluxo.

É possível calcular valorFinal quando o pagamento não foi aprovado? Não — pelo grafo dá pra ver que não existe nenhuma aresta ligando o bloco 6 ao bloco 7. Assim que o método entra no ramo verdadeiro da terceira decisão, ele já encerra ali.

Exercício 2 — Análise de leituras de temperatura

Código analisado

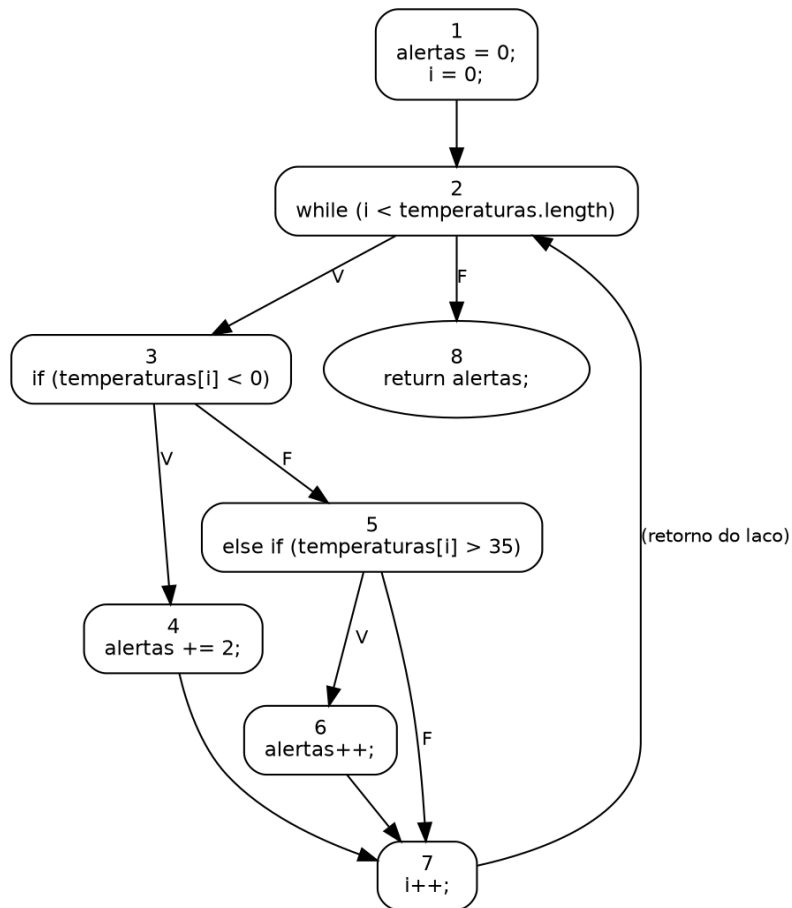
```
public int contarAlertas(double[] temperaturas) {  
    int alertas = 0;  
    int i = 0;  
  
    while (i < temperaturas.length) {  
        if (temperaturas[i] < 0) {  
            alertas += 2;  
        } else if (temperaturas[i] > 35) {  
            alertas++;  
        }  
  
        i++;  
    }  
  
    return alertas;  
}
```

1) e 2) Blocos básicos e decisões

- B1** — alertas = 0; i = 0; (inicialização, antes do laço)
- B2** — teste i < temperaturas.length (condição do while)
- B3** — teste temperaturas[i] < 0 (entrada do corpo do laço)
- B4** — alertas += 2; (ramo verdadeiro do if)
- B5** — teste temperaturas[i] > 35 (else if)
- B6** — alertas++; (ramo verdadeiro do else if)
- B7** — i++; (ponto de junção antes de voltar ao while)
- B8** — return alertas; (saída do laço)

As três decisões são: a condição do while (D1), o if temperaturas[i] < 0 (D2) e o else if temperaturas[i] > 35 (D3). O else if precisa ser representado como uma decisão à parte porque ele só é avaliado quando o if anterior dá falso, e tem suas próprias saídas verdadeira e falsa — ele não é a mesma decisão do if de cima, é um teste novo, encadeado.

3) e 4) Grafo de Fluxo de Controle



Grafo de fluxo de controle do método contarAlertas (V = verdadeiro, F = falso).

A aresta de retorno do laço é a que sai do bloco 7 (i++) e volta pro bloco 2 (teste do while) — ela é o que faz o laço realmente ser um laço no grafo, em vez de um fluxo reto.

5) e 6) Contagem de nós, arestas e complexidade ciclomática

N (número de nós) = 8

E (número de arestas) = 10

$$V(G) = E - N + 2 = 10 - 8 + 2 = 4$$

$$V(G) = \text{número de decisões} + 1 = 3 + 1 = 4$$

As duas fórmulas concordam.

7), 8) e 9) Base de caminhos independentes

- **P1** (nós 1-2-8) — entrada: temperaturas = { } (vetor vazio). Retorno esperado: 0
- **P2** (nós 1-2-3-4-7-2-8) — entrada: temperaturas = { -5.0 }. Retorno esperado: 2
- **P3** (nós 1-2-3-5-6-7-2-8) — entrada: temperaturas = { 40.0 }. Retorno esperado: 1
- **P4** (nós 1-2-3-5-7-2-8) — entrada: temperaturas = { 20.0 }. Retorno esperado: 0

P1 cobre a saída do laço sem nenhuma iteração (aresta 2→8). P2 exercita o ramo de temperatura negativa (arestas 3→4 e 4→7). P3 exercita o ramo de temperatura acima de 35 (arestas 3→5, 5→6 e 6→7). P4 exercita o caso em que a temperatura fica entre 0 e 35, inclusive, sem cair em nenhum dos dois alertas (aresta 5→7). No fim, as quatro rodadas juntas passam pelas 10 arestas do grafo.

10) Por que o retorno do laço precisa aparecer no CFG

Sem a aresta de retorno (7→2), o grafo não representaria a repetição de verdade — pareceria que o corpo do laço executa só uma vez e segue em frente. É essa aresta que fecha o ciclo e permite o

método processar mais de uma posição do vetor, então ela é obrigatória para o grafo refletir o comportamento real do while.

Questões para discussão

Um vetor com várias temperaturas percorre um único caminho ou repete partes do grafo? Ele repete partes do grafo. Cada posição do vetor faz o fluxo passar de novo pelos nós 2, 3, (4 ou 5/6) e 7, então a execução real é uma sequência longa que reaproveita os mesmos nós e arestas várias vezes — não conta como um caminho independente novo, é só o laço rodando mais vezes.

Qual entrada permite sair do método sem acessar uma posição do vetor? Um vetor vazio (`length = 0`). Nesse caso a condição do `while` já é falsa na primeira checagem e o método vai direto para o `return`, sem nunca indexar `temperaturas[i]`.

Os testes com 0 e 35 ajudam a avaliar quais fronteiras? O valor 0 testa a fronteira do primeiro `if` (`temperaturas[i] < 0`): zero não é negativo, então cai fora desse ramo. O valor 35 testa a fronteira do `else if` (`temperaturas[i] > 35`): como a comparação é estritamente maior, 35 também fica de fora do ramo de alerta. São justamente os valores de borda que costumam pegar erro de comparação (tipo usar `<=` no lugar de `<`).

Por que o `else if` deve ser representado como uma nova decisão? Porque ele é um teste condicional separado, com sua própria saída verdadeira e falsa, mesmo estando encadeado ao `if` anterior. Se ele não virasse um nó de decisão próprio no grafo, a terceira condição do método ficaria de fora da contagem e a complexidade ciclomática calculada ficaria errada (daria 3 em vez de 4).